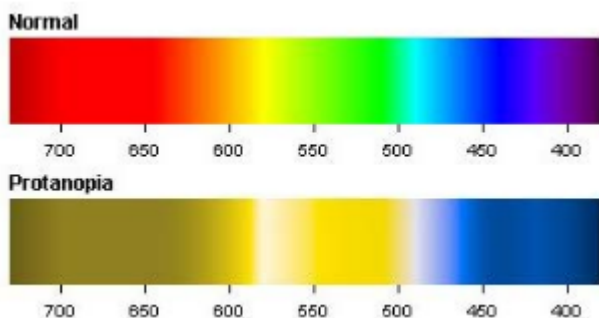


PSET5

1. Colorblindness Filters

El ojo humano contiene dos tipos de células fotorreceptoras (sensibles a la luz): bastones y conos. Los bastones son responsables de la visión en ambientes de poca luz, como por la noche, y los conos son responsables de la visión del color. Hay tres tipos de conos, cada uno responsable de un color particular, es decir, rojo, verde y azul. Los conos rojo, verde y azul trabajan juntos permitiéndote ver todo el espectro de colores. Por ejemplo, cuando los conos rojo y azul son estimulados de cierta manera, verás el color púrpura.

La condición conocida como daltonismo típicamente se manifiesta como una deficiencia en uno de estos tipos de conos. Por ejemplo, la protanopia es una falta de sensibilidad a la luz roja, por lo que lo siguiente es un ejemplo de la diferencia en la visualización de una imagen.



El objetivo de esta parte es crear filtros para simular estas diferencias. Para hacer esto, haremos uso de la **Librería de Imágenes de Python** o PIL para abreviar. De PIL importaremos `Image`, que es una sublibrería, o un conjunto de funciones y métodos, relacionados con el procesamiento de imágenes. Con `Image`, podemos convertir imágenes a una lista de píxeles y manipular los valores RGB (rojo, verde, azul) de estos píxeles.

Podemos replicar los efectos del daltonismo con una multiplicación matricial entre una matriz de transformación de daltonismo y los valores RGB de un píxel particular, que se representan como un vector con 3 entradas.

No necesitas saber nada sobre multiplicación matricial; sin embargo, si deseas obtener más información al respecto, consulta [aquí](#).

Aquí hay un desglose general del proceso de transformación de imagen:

1. Abrir la imagen en Python
2. Recuperar una lista (de tuplas o enteros) de la información de píxeles
3. Iterar a través de los píxeles y multiplicarlos por la matriz de transformación usando `matrix_multiply`
4. Guardar los píxeles transformados como una imagen.

1.1 Implementing Helper Functions

1.1.1 `img_to_pix(filename)`

La primera función auxiliar será convertir la imagen a una lista de píxeles. La entrada es una cadena que representa un archivo de imagen, como 'example.jpg', y la salida es una lista correspondiente a los píxeles de esa imagen, por ejemplo: `[(0,0,0),(255,255,255),(38,29,58)...]` para RGB, `[60, 66, 72,...]` para B&W. La lista consiste en n-tuplas que corresponden a los valores RGB de ese píxel para una imagen RGB, o un entero correspondiente al brillo de ese píxel para una imagen B&W.

Querrás abrir la imagen y obtener los datos de esa imagen. (Consulta la documentación del módulo Image para saber cómo hacer estas cosas.) Nota: No te preocupes por determinar si una imagen es RGB o B&W. Las funciones de la biblioteca PIL que usarás devuelven los valores correctos de píxeles para cualquier modo de imagen.

Esto debería ser **muy corto** si puedes encontrar las funciones apropiadas.

Mi solución

```
1  def img_to_pix(filename):
2      im = Image.open(filename)
3      if im.mode == 'RGB':
4          im = im.convert("RGB")
5          im = list(im.getdata())
6          return im
7      elif im.mode == 'L':
8          im = im.convert("L")
9          im = list(im.getdata())
10         return im
```

1.1.2 `pix_to_img(pixels_list, size, mode)`

La siguiente función auxiliar es convertir una lista de píxeles a una imagen. La entrada será `pixels_list`, en el mismo formato que la salida

de la función anterior, un parámetro `size` que es una tupla de dos elementos que describe las dimensiones de la imagen de salida, y un parámetro `mode` que determina si los píxeles de entrada representan una imagen en blanco y negro o a color. Supón que `size` es una entrada válida tal que `size[0] * size[1] == len(pixels)`.

En otras palabras, quieres **copiar valores de píxeles de un objeto secuencia a la imagen**, por lo que deberías consultar la documentación del módulo Image sobre cómo hacer esto. Consulta la cadena de documentación para obtener detalles sobre cómo debe representarse el modo.

Esto debería ser **muy corto** si puedes encontrar las funciones apropiadas.

Mi solución

```
1 def pix_to_img(pixels_list, size, mode):
2     img = Image.new(mode, size)
3     img.putdata(pixels_list)
4     return img
```

1.1.3 `filter(pixels_list, color)`

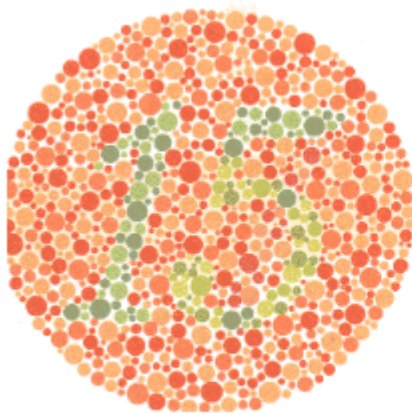
Esta función toma una lista de píxeles en formato RGB, como `[(0,0,0), (255,255,255),(38,29,58)...]`, así como un color: 'red', 'blue', 'green', o 'none'. El propósito de esta función es aplicar una transformación a los píxeles en la lista de entrada que simule el deterioro en los conos del color de entrada. Devuelve la lista de píxeles transformados.

Para realizar la transformación, multiplica cada píxel por la matriz apropiada. Hemos proporcionado una función `make_matrix` que suministrará una matriz que representa la transformación apropiada dependiendo de la cadena de entrada. Por ejemplo, `make_matrix('red')` devolverá la matriz que representa una deficiencia roja en la visión de uno.

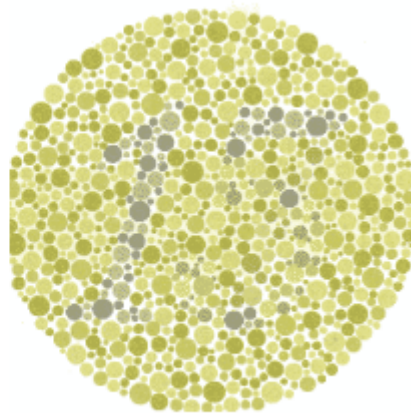
Recuerda que debes **iterar a través de la lista de píxeles y multiplicar cada uno por la matriz apropiada**. Usa el método `matrix_multiply(matrix1, matrix2)` proporcionado para hacer esto. Ten en cuenta que queremos que **la matriz de transformación sea el primer argumento y el vector de píxel RGB sea el segundo**, o de lo contrario la imagen resultante será incorrecta. Ten en cuenta que `matrix_multiply` devuelve una lista de flotantes y que los píxeles **deben ser tuplas de enteros**.

Esto debería ser **bastante corto** si puedes encontrar las funciones apropiadas.

Si todo se hace correctamente, entonces ejecutar el programa con la imagen de prueba ('image_15.png') y una deficiencia roja debería resultar en lo siguiente:



The original image



The transformed image

Imágenes como esta son conocidas como el test de Ishihara.

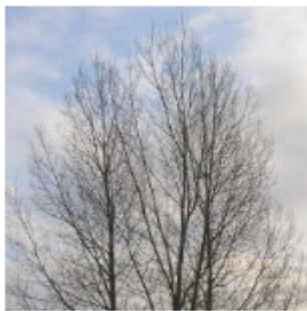
Mi solución

```
1 def filter(pixels_list, color):
2     pixels = []
3     matrix = make_matrix(color)
4     for p in pixels_list:
5         transformed = matrix_multiply(matrix, p)
6         if isinstance(transformed, (tuple, list)):
7             transformed = tuple(max(0, min(255, int(round(x)))) for x in
transformed)
8         else:
9             transformed = tuple(max(0, min(255, int(round(transformed))))))
10        pixels.append(transformed)
11    return pixels
```

2. Hidden Messages in Images

Aquí tienes la traducción al español:

En una imagen, es posible ocultar una segunda imagen (secreta) que solo puede ser recuperada mediante ciertas operaciones matemáticas. Esto se conoce como esteganografía. En esta sección, repasarás algunos ejemplos y escribirás tu propio módulo de esteganografía para ocultar imágenes.



The original image



The secret image

Un ejemplo de esteganografía se proporciona arriba. La imagen de la izquierda contiene la imagen de la derecha. Cuando termines con este conjunto de problemas, entenderás cómo extraer la imagen secreta de la original.

En una imagen en blanco y negro, cada píxel está representado por un valor numérico que indica su brillo. El valor mínimo para cada píxel es 0 (que corresponde al negro), mientras que el valor máximo (que corresponde al blanco) depende del número de bits utilizados para cada píxel. En nuestros ejemplos, usaremos imágenes de 8 bits. Esto significa que los píxeles pueden tomar valores entre 0 y 255, inclusive (¿puedes ver por qué?).

Para esta parte, asume que estás trabajando con imágenes que tienen profundidad de color de 8 bits.

2.1 Trabajando con Números Binarios

Esta parte del conjunto de problemas se basa en leer y manipular números binarios.

Como ejemplo, echemos un vistazo a la representación binaria del número decimal 13, que es 1101. Comenzando desde el dígito más a la derecha en un número binario y moviéndose hacia la izquierda, cada posición de índice representa una potencia creciente de 2:

$$\begin{array}{cccc} 1 & 1 & 0 & 1 \\ 2^3 & 2^2 & 2^1 & 2^0 \end{array}$$

Cuando sumamos estas potencias de 2 (es decir, $2^3 + 2^2 + 2^0$), obtenemos 13.

2.1.1 Bits Menos Significativos

La técnica más común para incrustar imágenes secretas es usar el **bit menos significativo** (LSB) de cada valor de píxel. Podemos modificar el LSB en cada píxel sin hacer ninguna diferencia notable. De esta

manera, una imagen puede llevar otra imagen secreta que es completamente invisible al ojo desnudo.

¿Qué es un bit menos significativo?

Ejemplo: ¿Cuál es el LSB de 13? Primero convierte la representación decimal a binario como se muestra arriba: $13 \rightarrow 1101$.

El LSB es el dígito **más a la derecha**, en este caso un 1. Los 3 LSBs son los dos dígitos más a la derecha, en este caso 101, que es 5 en base 10.

2.1.2 Preguntas a Considerar

Nota: estas no son parte de la tarea, son meramente para ayudarte a pensar a través del problema.

1. Dado que un número es divisible por 2, ¿Cuál sería el valor del LSB? ¿Cuáles de estos números binarios son divisibles por 2: 1001, 10111, 10110, 111110?
2. ¿Cómo podemos obtener el valor del LSB sin convertir un número base 10 a representación binaria?
3. ¿Sería el número binario xxx0 (x puede ser 0 o 1) divisible por 4? ¿Si es así, por qué?
4. ¿Cuál sería el residuo de estos números binarios cuando se dividen por 4: xxx01, xxx10, xxx11?
5. ¿Cómo podemos extraer el LSB de un número base 10 sin convertirlo a binario?
6. ¿Cómo podemos extraer los n LSBs de un número base 10 sin convertirlo a binario?

Por favor asegúrate de entender estas preguntas antes de continuar. Entenderlas simplificará tu implementación de `extract_end_bits`.

2.2 Implementando `extract_end_bits`

Para extraer números arbitrarios de estos LSBs de valores de píxeles, implementarás una función auxiliar `extract_end_bits(num_end_bits, pixel)`. Esta función devolverá los **num_end_bits LSBs de pixel como un entero en base 10**.

Parámetros:

- **num_end_bits:** el número de LSBs a devolver
- **pixel:** el valor del píxel cuyos LSBs serán devueltos

Por ejemplo, basándose en el ejemplo anterior, podríamos escribir:

```
1  extract_end_bits(1, 13) # obtener un LSB -> devolver 1
2  extract_end_bits(2, 13) # obtener dos LSBs -> devolver 1
3  extract_end_bits(3, 13) # obtener tres LSBs -> devolver 5
```

Pistas:

- No conviertas números a binario.
- La implementación de esta función debería ser muy corta.
- Al pensar sobre cómo implementar `extract_end_bits`, el operador módulo (%) puede ser muy útil.
- Puedes ejecutar `test_ps5_student.py` para verificar tu implementación de `extract_end_bits`.

Mi solución

```
1  def extract_end_bits(num_end_bits, pixel):
2      if isinstance(pixel, tuple):
3          converted = []
4          for i in pixel:
5              converted.append(i % 2 ** num_end_bits)
6          return tuple(converted)
7      else:
8          return pixel % 2 ** num_end_bits
```

2.3 Recuperando una imagen binaria

En una imagen binaria (blanco y negro), cada píxel está representado por un valor numérico que representa su intensidad. El valor mínimo para cada píxel es 0 (que corresponde al negro), mientras que el valor máximo (que corresponde al blanco) depende del número de bits utilizados para cada píxel. En nuestros ejemplos, usaremos imágenes de 8 bits. Esto significa que los píxeles pueden tomar valores entre 0 y 255, inclusive (¿puedes ver por qué?).

En `ps5.py`, completa la parte en blanco y escribe la porción de la función `reveal_image` implementando `reveal_bw_image` de acuerdo a la cadena de documentación. Esta función toma el nombre de archivo de una imagen, usa el valor LSB para encontrar la imagen oculta, y la devuelve como un objeto `PIL.Image`.

Nota: `reveal_image` debería poder manejar tanto imágenes BW como RGB; implementarás la porción RGB en la parte c.

Importante: La imagen oculta está incrustada en el bit menos significativo y puede ser recuperada usando `extract_end_bits` con el

`num_bits` apropiado. Sin embargo, como las operaciones PNG pueden producir valores muy de bajo contraste que no puedes ver, la información, deberías realizar otra operación para reescalar los valores de los píxeles, para que puedan hacer uso del rango completo de valores disponibles.

En la carpeta del conjunto de problemas, encontrarás una imagen llamada `hidden1.bmp`. Usando tu función, encuentra la imagen secreta en este archivo y guarda una copia de la misma mediante la `PIL`. Para mostrar la imagen usando Python, la imagen secreta también debería ser en escala de grises. Recuerda qué nombre de archivo necesitarás un poco.

****Nota:** estas no son parte de la tarea, son meramente para ayudarte a pensar a través del problema.

1. ¿Qué números en base 10 pueden ser representados en 1 bit? ¿En 2 bits?
2. ¿Cuál es el rango de valores de un píxel BW (o un elemento en un píxel RGB)?
3. ¿Cómo podemos reescalar valores LSB para aprovechar el rango completo de un píxel?

Mi solución

```
1 def reveal_bw_image(filename):
2     im = img_to_pix(filename)
3     width, height = Image.open(filename).size
4     result = []
5     for i in im:
6         p = extract_end_bits(1, i) * 255
7         result.append(p)
8     return pix_to_img(result, (width, height), 'L')
```

2.4 Recuperando una imagen a color

Los métodos descritos arriba son lo suficientemente robustos para ser aplicados a más bits. En esta parte, trabajarás con una imagen RGB y usarás `tres` bits menos significativos para recuperar la imagen secreta.

En `ps5.py`, completa el componente relevante de la función `reveal_image` implementando `reveal_color_image` de acuerdo a su cadena de documentación. Para una imagen a color, esta función toma el nombre del archivo, extrae la imagen secreta de los tres bits menos

significativos de cada canal (rojo, verde y azul) y la devuelve como un objeto `PIL.Image`. Nuevamente necesitarás reescalar los píxeles.

En la carpeta del conjunto de problemas, encontrarás una imagen llamada `hidden2.bmp`. Usando tu función, encuentra la imagen secreta en este archivo y guarda una copia de la misma usando la funcionalidad de la biblioteca PIL. Recuerda qué nombre le pusiste al archivo ya que necesitarás el nombre del archivo en un momento.

Mi solución

```
1 def reveal_color_image(filename):
2     im = img_to_pix(filename)
3     width, height = Image.open(filename).size
4     result = []
5     for i in im:
6         p = extract_end_bits(3,i)
7         result.append((p[0]*255 // 7, p[1]*255 // 7, p[2]*255 // 7))
8     return pix_to_img(result, (width, height), 'RGB')
```

2.5 Haciendo Tu Arte Propio

En este punto deberías tener tres nuevas imágenes:

1. Filtered image_15.png
2. Unhidden hidden1.bmp
3. Unhidden hidden2.bmp

Usando la función auxiliar proporcionada `draw_kerb`, ejecuta cada una de tus imágenes a través de esta función para agregar tu marca de agua kerberos a cada una.

Para comenzar, si tuvieras una imagen llamada `img1.png` y tu kerberos fuera `timthebeaver` podrías llamar `draw_kerb("img1.png", "timthebeaver")` lo cual produciría una imagen en tu directorio llamada `img1_kerb.png`.

Mi solución

```
1 draw_kerb("Filtered image_15.png", "matiaspetenatti")
2 draw_kerb("Unhidden hidden1.bmp", "matiaspetenatti")
3 draw_kerb("Unhidden hidden2.bmp", "matiaspetenatti")
```